

SESSION 3 OF 3 • TEST IT

DOES IT ACTUALLY WORK?

You built a strategy. Now we find out, with the only honest tool there is: the past.

REMEMBER THIS MORNING?

The flip strategy. Profitable or loss-making?

Red → green: buy the ATM call. Green → red: buy the ATM put. It placed real sandbox orders two hours ago.

PROFITABLE

Hands up.

LOSS-MAKING

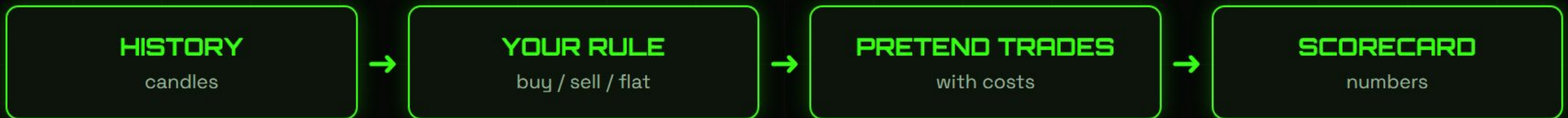
Hands up.

Honest answer: **nobody in this room knows yet.**

One afternoon of orders proves nothing. Finding out is exactly what backtesting does.

DEFINITION

Run your rule on the past.
Count what **would have** happened.



Return · how much your money grew

$$\text{return} = (\text{exit} - \text{entry}) \div \text{entry}$$

what you got out, minus what you put in, as a share of what you put in

The engine does the same sum on the whole portfolio, after costs: $(\text{end} \div \text{start}) - 1$. ₹10,00,000 → ₹11,50,000 is +15%.

WORKED EXAMPLE · ZOMATO, SESSION 1

- 1 **Entry price:** bought at ₹58 a share (26 Apr 2023)
- 2 **Exit price:** sold at ₹184 a share (27 May 2024)
- 3 **Subtract** what you paid from what you got: $184 - 58 = ₹126$ profit per share
- 4 **Divide** by what you paid: $126 \div 58 = 2.17$
- 5 **As a percentage:** **+217%** (a ₹58 stake became ₹184)

CAGR · growth per year

$$\text{CAGR} = (\text{end} \div \text{start}) ^ { (1 \div \text{years}) } - 1$$

the one steady yearly rate that would have produced the same end result

Zomato again

₹58 → ₹184 in 13 months = 3.17x. $3.17 ^ { (1 \div 1.09) } - 1 \approx +190\%$ a year. Nobody repeats that; it is one lucky year, annualised.

WORKED EXAMPLE · ₹10,00,000 → ₹14,40,000 IN 2 YEARS

- 1 **End ÷ start:** $14,40,000 \div 10,00,000 = 1.44$ (the money grew 1.44x)
- 2 **1 ÷ years** = $1 \div 2 = 0.5$, so take the **square root:** $1.44 ^ { 0.5 } = 1.20$
- 3 **Minus 1:** $1.20 - 1 = +20\%$ a year
- 4 **Not** $44 \div 2 = 22\%$. Year two grew on top of year one's gains. That is compounding.
- 5 **Check:** $10,00,000 \times 1.2 \times 1.2 = 14,40,000 \checkmark$

Zomato: 26 Apr 2023 to 27 May 2024 is 397 days ≈ 1.09 years; $184 \div 58 = 3.17$. The engine uses bars instead of years: $(1 + \text{return}) ^ { (246 \div \text{bars}) } - 1$, so a 3-month test and a 12-month test compare.

Max drawdown · the worst fall



$$\text{drawdown} = (\text{trough} - \text{peak}) \div \text{peak}$$

the biggest such fall in the whole test is the max drawdown

WORKED EXAMPLE · PAYTM, SESSION 1

- 1 Peak:** the highest your money reached. Say you bought at the top: ₹785 (Aug 2022)
- 2 Trough:** the lowest point after it: ₹317 (8 May 2024)
- 3 Subtract** the peak from the trough: $317 - 785 = -₹468$
- 4 Divide** by the peak: $-468 \div 785 = -59\%$ max drawdown
- 5** To get back to ₹785 from ₹317 you need **+148%**. Could you have held through that?

Paytm daily closes, NSE via the Nubra SDK (data/csv/PAYTM.csv): ₹784.70 on 17 Aug 2022 (the Session 1 buy), lowest close ₹317.14 on 8 May 2024; it later recovered to ₹1,277 (15 Oct 2025).

Sharpe ratio

The Sharpe ratio measures the **excess return** of an investment **per unit of its total risk** (volatility).

$$\text{Sharpe} = (R_p - R_f) \div \sigma_p$$

R_p = return of the strategy · R_f = risk-free return (a government bond; 0 in our engine) · σ_p = standard deviation of the returns: the wobble

- 1 Daily numbers**, five days: +1.2%, -0.9%, +0.4%, -1.1%, +0.7%. Average R_p = **0.06%** a day; wobble σ_p = **1.01%** a day.
- 2 Divide**: $(0.06 - 0) \div 1.01 = \mathbf{0.059}$. That is a *daily* Sharpe.
- 3 Make it yearly**: returns add up over 246 trading days, but the wobble only grows with $\sqrt{246} = \mathbf{15.7}$. So daily Sharpe $\times \sqrt{246}$: $0.059 \times 15.7 = \mathbf{\text{Sharpe} \approx 0.93}$
- 4** That is the only reason you see a $\sqrt{}$ in our scorecard: **same formula**, computed on daily returns, then annualised.



Below 0: losing · about 1: decent · above 2: suspicious · above 3: check for look-ahead

Named after William F. Sharpe (1966). Engine: `mean(returns) ÷ std(returns) × √(bars per year)`, risk-free 0 (`backtest.py`, `metrics`). Intraday uses `√(bars per year)` instead of `√246`.

Trades and win rate

Number of trades

Too few and the result is luck.
Minimum **10** to qualify today.

Win rate

Share of trades that made money. 35% can still be profitable if winners are big.

Exposure

How much of the time you were in the market. 20% exposure with +5% is different from 100% with +5%.

Costs eat strategies

- ◆ **Brokerage:** a fee per order (₹20 in today's engine)
- ◆ **Slippage:** you never get the price you saw (0.05% today)
- ◆ 300 trades × both = a strategy that "worked" now loses

Rule

If a backtest ignores costs, it is a story, not a test.

In-sample vs out-of-sample

In-sample

The data you looked at while building the rule. You tuned it here, so of course it works here.

Out-of-sample

Data the rule has **never seen**. The only result that means anything.

Today's data stops at **30 June 2026**. July to September is locked away. **That is where you will be judged.**

Timeframe · how big is one bar?

1d

1h

30m

15m

5m

1m

Daily

246 bars a year. Slow, cheap, readable. The default today.

Intraday

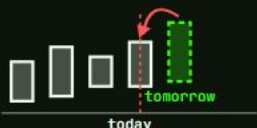
Thousands of bars a year. More trades, more costs, more noise. Data from Sep 2025.

Your choice

One line in your file: `TIMEFRAME = "15m"`. The engine loads the right data and annualises correctly.

FIVE WAYS THE PAST FOOLS YOU

Why most backtests lie



Look-ahead

Using information you would not have had yet.

REAL EXAMPLE

A rule that peeks one day ahead, on our 52 stocks: **+316%** in a year, **99%** of trades win. Impossible in real life. Our engine blocks it.



Overfitting

Tuning a rule until it fits the past perfectly.

REAL EXAMPLE

We tried **41** moving-average pairs on Jul-Dec 2025. Best one: Sharpe **2.01**. Same rule on Jan-Jun 2026: Sharpe **-1.87**.



Survivorship

Testing only the winners that are still around.

REAL EXAMPLE

Our own data uses **today's** NIFTY 50. Stocks that fell out during the year are missing, so every result here is a little too rosy.



Costs Ignored

Forgetting that every trade costs money.

REAL EXAMPLE

A student's 5-minute breakout: **+3%** before costs, **-35%** after. 21,707 trades, about ₹89 each.



Data snooping

Trying many ideas and showing only the lucky one.

REAL EXAMPLE

50 coin-flip strategies on our data: the luckiest made **+15%**, Sharpe 1.43. Pure chance. 12 of 50 beat our best real example.

Examples computed on the repo data (52 stocks, Jul 2025-Jun 2026) with the engine and engine-validation/VALIDATION.md; costs = brokerage Rs 20 or 0.03% + 0.03% slippage.

Educational purposes only. Not investment advice. Backtest results are hypothetical and do not predict future returns.

Aryan · Nubra

KEEP THIS

The honest-backtest **checklist**

✓ **Next-bar execution**

decide on this bar, trade on the next

✓ **Costs and slippage in**

₹20 per order + 0.05%

✓ **Out-of-sample holdout**

a quarter you never see

✓ **Enough trades**

10 minimum, more is better

Today's engine does all four for you. Same four boxes as the definition slide. **Now yours.**

COMPETE · THE REPO

github.com/aaryan-say/bmsit-quant-101

```
git clone https://github.com/aaryan-say/bmsit-quant-101.git # 1 get it
cd bmsit-quant-101 # 2 go in
pip install -r requirements.txt # 3 pandas + matplotlib
python backtest.py strategies\ma_crossover.py # 4 run the example
```

The last line printed is your **SUBMIT scorecard**:

```
SUBMIT | ma_crossover | tf=1d | sharpe=-1.29 | cagr=-8.8% | maxdd=-11.6% | trades=292
```

No git?

GitHub → Code → Download ZIP,
unzip, open a terminal in the
folder.

Windows one-click

`run.bat` makes a venv, installs,
runs the example.

mac / Linux

Same four commands; use
`python3` and
`strategies/ma_crossover.py`.

Repo: <https://github.com/aaryan-say/bmsit-quant-101> (public). SUBMIT line: output of `python backtest.py --all` on the repo data, 8 Oct 2026; the format is `SUBMIT | name | tf | sharpe | cagr | maxdd | trades`.

Educational purposes only. Not investment advice. Backtest results are hypothetical and do not predict future returns.

Aryan · Nubra

WHAT YOUR FILE MUST LOOK LIKE

One function. That's the whole contract.

```
TIMEFRAME = "1d"                # or "1h", "15m", ...

def strategy(df):
    # df: date, open, high, low, close, volume (oldest first)
    fast = df["close"].rolling(20).mean()
    slow = df["close"].rolling(50).mean()
    return (fast > slow).astype(int)    # +1 long, 0 flat, -1 short, per row
```

In

one stock's candles

Out

a position for every row

Engine

runs it on all 52, equal weight,
next-bar fills, costs, scorecard

WHAT YOU GET BACK

The scorecard, on real data

STRATEGY	TF	SHARPE	CAGR	MAX DD	TRADES
zscore_meanrev	1d	0.31	+1.8%	-7.2%	214
ma_crossover	1d	-1.18	-8.2%	-11.3%	292
breakout	1d	-1.60	-11.2%	-12.1%	335
first_green_15m	15m	-2.15	-9.9%	-8.5%	4,805
orb_vwap_5m	5m	-26.8	-41.6%	-35.1%	21,707

The examples in the repo, July 2025 to June 2026, 52 stocks, costs included. One of five makes money.

Output of ``python backtest.py --all`` on the repo data, 9 Oct 2026 (costs: brokerage Rs 20 or 0.03%, whichever lower, + 0.03% slippage).

Educational purposes only. Not investment advice. Backtest results are hypothetical and do not predict future returns.

Aryan · Nubra

BEFORE YOU START: THIS IS A TOUGH CHALLENGE

Why most strategies **lost** this year

1 · The market went
nowhere

+0.1%

Buy-and-hold of all 52 stocks for the whole year. Strategies that need a trend (crossovers, breakouts) had none to ride.

2 · Every trade costs
money

₹89

What one round trip costs on a ₹1 lakh position (brokerage + slippage). The 5-minute breakout earned ₹7 per trade before costs.

3 · Chop rewards patience

0.31

The only winner bet on prices returning to normal (mean reversion), and traded rarely. That is the Sharpe to beat.

Fewer, better trades. Fit the idea to the market you have. **Beat 0.31 and you have done something real.**

Buy-and-hold, same universe, 1 Jul 2025 to 30 Jun 2026 (daily): +0.12%. ORB cost decomposition from engine-validation/VALIDATION.md (Rs 7.3 gross per trade vs Rs 89 cost per round trip).

Educational purposes only. Not investment advice. Backtest results are hypothetical and do not predict future returns.

Aryan · Nubra

THE COMPETITION

Rules of the game

1 • 25 minutes

Submit any time; resubmit freely, your **latest** entry counts. Showdown **closes submissions itself at 00:00**: nothing is accepted after.

2 • Submit in Showdown

Same link as the quizzes. Paste your **whole .py file** and the **SUBMIT line** it printed.

3 • We run your file

Your file is re-run automatically on **July to September 2026**, data nobody has seen. That result is your score, not the number you paste.

4 • Judged on Sharpe

Highest hidden Sharpe wins. At least **10 trades**. Tie: the earlier submission wins.

5 • Not eligible if

your SUBMIT line doesn't match your file, it peeks at the future, it crashes, or it imports anything beyond pandas and numpy.

6 • Reward

The student who cracks the best strategy gets a reward.

Judge: safety check of the code, re-run on public data (must match your SUBMIT line), then the hidden-data run. Showdown "Strategy Showdown" round opens with the clock.

CLOCK STARTS NOW • 25 MINUTES

Five steps. Submit before the bell.

1 Clone

```
git clone https://github.com/aaryan-say/bmsit-quant-101.git · cd bmsit-quant-101 · pip install -r requirements.txt
```

2 Prompt

Paste `PROMPT.md` into ChatGPT, add your idea at the bottom, save the reply as `strategies\mine.py`

3 Run

```
python backtest.py strategies\mine.py
```

 · read the scorecard: Sharpe, drawdown, trades

4 Iterate

Change **one** thing, run again. Keep it only if Sharpe improved and trades stayed ≥ 10 . At least three rounds.

5 Submit in Showdown

Paste the whole file + its `SUBMIT` line. Wait for the green "Checked" on your screen. Resubmit as often as you like.

25:00

START

press again to reset · bell at zero

Nothing printed?

From the **repo folder**: `py backtest.py strategiesmine.py` (Windows: `py`, not `python`)

Stuck on an idea?

Change one number in an example file and see what moves.

"Doesn't match"?

Re-run that exact file, copy the new `SUBMIT` line, resubmit.

PENS DOWN

Now the data you **never saw**

- ◆ Your scores were on July 2025 to June 2026
- ◆ Every file was already run on **July to September 2026**
- ◆ Same engine, same costs, one flag: `--data data\hidden`
- ◆ If the rule is real, it survives. If it memorised, it does not.



ALREADY JUDGED WHILE YOU WORKED

How the **reveal** works

- 1 Every file was judged the moment it arrived**
Safety check, re-run on public data against your SUBMIT line, then a run on **July to September 2026**. Same engine, same costs.
- 2 Not eligible: out**
Mismatched SUBMIT line, look-ahead, crash, unsafe code, or fewer than 10 trades.
- 3 The top three, read on screen**
Before announcing, we open the top three files in front of you. A human check no script replaces.
- 4 The board and the winner**
Ranked by hidden Sharpe. Expect the order to change from what you saw. Winner card: name and USN. Come to the stage.

Hidden set: the same 52 symbols, 1 Jul to 30 Sep 2026, kept off the public repo (data/hidden, git-ignored). Tie-break: the earlier submission.

Educational purposes only. Not investment advice. Backtest results are hypothetical and do not predict future returns.

Aryan · Nubra

SHOWDOWN · STRATEGY ROUND

STRATEGY SHOWDOWN

In-sample board first. Then one button: [run on hidden data](#).

backboard-crafter-closure.ngrok-free.dev

Same login as the quizzes. The winner card shows name and USN.

THREE SESSIONS, ONE QUESTION

Did you **learn** something today?



Are you an **algo trader** now?

A **quant developer**?

- ◆ **Session 1:** you saw the market the way a machine sees it
- ◆ **Session 2:** you built a strategy and a screener that placed sandbox orders
- ◆ **Session 3:** you tested it honestly, on data it never saw

IF THIS CLICKED

A **path**, not a promise

1 • Maths

Statistics and probability, properly. Linear algebra. That is the real moat.

2 • Code

Python and pandas fluently. Then one systems language. Git from day one.

3 • Build in public

Backtests, tools, contributions. A GitHub profile beats a CV line.

4 • Markets

Read, paper trade, keep a journal. Microstructure before prediction.

Roles exist for builders, data engineers, researchers and traders. You do not have to trade to work here.

KEEP BUILDING

Three repos, open for you

bmsit-quant-101

Your strategy →

`strategies/community/` via PR.

The Session 2 screener and dashboard live in `session2/`.

nubra-python-sdk

105 examples. Add one from today.

nubra-backtest-engine

Options backtesting. Issues welcome.



github.com/aaryan-say/bmsit-quant-101 · github.com/NubraAPI/nubra-python-sdk (MIT, 105 examples) · github.com/aaryan-say/nifty-options-delta-neutral-backtest (PyPI: nubra-backtest-engine)

Educational purposes only. Not investment advice. Backtest results are hypothetical and do not predict future returns.

Aryan · Nubra

LAST TWO MINUTES

Thank you. Tell us what to fix.



Feedback

Scan, two minutes, honest. It decides what next year's room gets.

forms.gle/iQSPKL8QFtLwESSQ8

Docs and code

nubra.io/products/api/docs ·
github.com/NubraAPI · github.com/aaryan-say/bmsit-quant-101

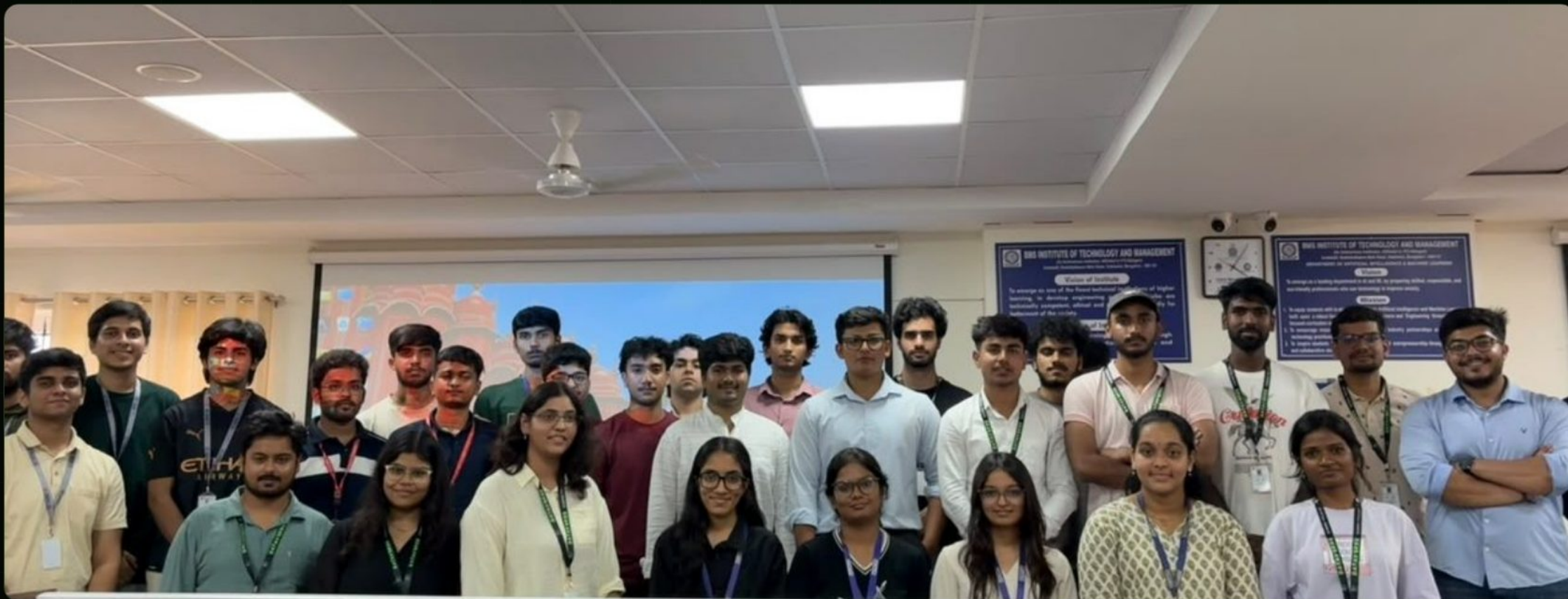
Say hi

DataVerse: [dataverse_bmsit](#) · the volunteers in the room



9 OCTOBER 2026 · BMSIT, BENGALURU

The room



Quant Engineering 101 · DataVerse x Nubra · BMS Institute of Technology and Management, Bengaluru

Educational purposes only. Not investment advice. Backtest results are hypothetical and do not predict future returns.

Aryan · Nubra

THE DAY IN PICTURES

See it. Build it. **Test it.**

